



Today's agenda

↳ Prototype design pattern



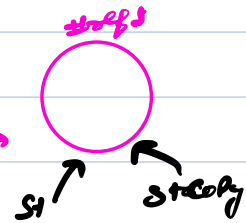
AlgoPrep



* Given an object of a class, we need to create multiple copies of the given object.

// idea 0

```
Student st = new Student();  
Student stCopy = st;
```



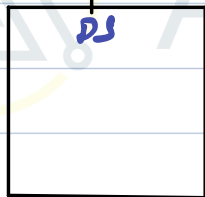
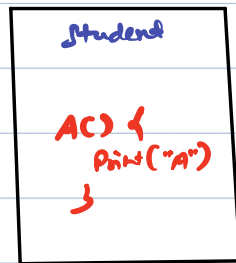
// idea 1

```
Student st = new Student();  
Student stCopy = new Student();
```

```
stCopy.name = st.name;  
stCopy.age = st.age;  
stCopy.weight = st.weight;  
:  
:
```

Cons:

1) Client needs to know all the internal details of Student class.



→ JRP x
→ OCP x

```
Student st = new Student() / IS();
```

↓

```
Student stcopy = _____
```

```
if (instance of st == Student) {  
  stcopy = new Student();  
}
```

```
else if (instance of st == IS) {  
  stcopy = new IS();  
}
```

```
else if (instance of st == DS) {  
  stcopy = new DS();  
}
```

```
}
```



Idea2

↳ copy constructors

→ now client doesn't need to know all the internal details of class.

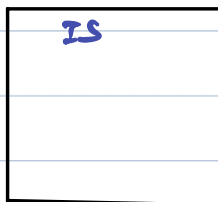
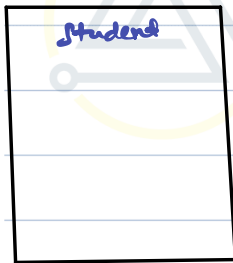
Class Student {

```
Student (Student original) {  
    this.name = original.name;  
    this.age = original.age;  
    ;  
}
```

Student st = new Student() / IS;

↓

Student stcopy = _____



```
if (instance of st == Student) {  
    stcopy = new Student(st);  
}  
else if (instance of st == IS) {  
    stcopy = new IS(st);  
}
```

Note: All the child class must have the copy constructor.



Idea 3

↳ instead of having copy constructor, have a copy method and return the object.

```
Student
{
    Student original;
    Student copy c;
    Student s = new Student();
    s.name = orig.name;
    return s;
}
```

```
Student st = new Student()/IS();
```

```
Student stcopy = st.Copy();
```

```
IS
{
    IS copy (...);
    IS ist = new IS();
    ;
}
```

→ OCP is not violated ✓

→ now client doesn't need to know all the internal details of class.

→ To enforce copy method use interface.



→ Search API

AlgoPrep.in / Course-dashboad / Systendesign / Query

accessing
notu → accessing
video
+
accessing
codes

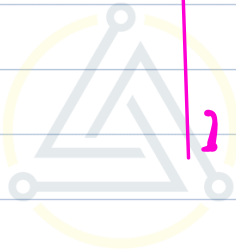
SearchAPI <

url =

Path = - - -

Auth: - - -

query: - - -



AlgoPrep



→ there are scenarios where we don't want to create object from scratch, rather we copy of existing object and change few required attributes.

Class Student {

name

age

weight

batch

instructor

}

Student obj = new Student();

obj.name = "Roshan";

obj.name = 27;

obj.weight = 72;

obj.batch = SD2

obj.instructor = "Abhishek"

SD2 → original obj 1

SD3 → original obj 2

PSA1 → original obj 3

Registry

Map<String, Student> map;

register (key, student) {

map.put (key, student);

}

Student get (key) {

return map.get (key);

}

Client will first call registry and then clone the objects.



Break till 9:20 PM

Code

```
package ProtoType;

public class Student implements Prototype<Student> {

    private String name;
    private int age;
    private int weight;
    private String batch;
    private String Instructor;

    • public Student clone() {
        Student stCopy = new Student();

        stCopy.name = this.name;
        stCopy.age = this.age;
        stCopy.weight = this.weight;
        stCopy.batch = this.batch;
        stCopy.Instructor = this.Instructor;

        return stCopy;
    }

    • public String getName() {
        return name;
    }

    • public void setName(String name) {
        this.name = name;
    }

    • public int getAge() {
        return age;
    }
}
```

```
1 package ProtoType;
2
3 public interface Prototype<T> {
4
5     T clone();
6 }
7 |
```



```
3 import java.util.HashMap;
4
5 public class StudentRegistry {
6     HashMap<String, Student> map = new HashMap<>();
7
8     void register(String key, Student st) {
9         map.put(key, st);
10    }
11
12    Student get(String key) {
13        return map.get(key);
14    }
15
16    public void fillRegistry(StudentRegistry studentregistry) {
17        Student obj1 = new Student();
18        obj1.setBatch("SD2");
19        obj1.setInstructor("Abhishek");
20        studentregistry.register("SD2", obj1);
21
22        Student obj2 = new Student();
23        obj1.setBatch("SD3");
24        obj1.setInstructor("Abhishek");
25        studentregistry.register("SD3", obj2);
26    }
27 }
28
29 }
```

```
public class Client {
    public static void main(String[] args) {
        StudentRegistry studentregistry = new StudentRegistry();
        studentregistry.fillRegistry(studentregistry);

        Student Preetham = studentregistry.get("SD2").clone();
        Preetham.setAge(20);

        Student Sahil = studentregistry.get("SD2").clone();
        Sahil.setAge(25);

    }
}
```